

# HiSpTRSV: Exploring Tile-Level Parallelism for SpTRSV Acceleration on FPGAs

Fan Sun, Fang Dong\*, Dian Shen  
School of Computer Science and Engineering  
Southeast University  
{fansun, fdong, dshen}@seu.edu.cn

**Abstract**—Sparse Triangular Solve (SpTRSV) is a critical level-2 kernel in sparse Basic Linear Algebra Subprograms (BLAS). While Field-Programmable Gate Array (FPGA) accelerators for SpTRSV focus on optimizing individual tiles, they overlook inter-tile parallelism. Designing an inter-tile parallelism accelerator poses challenges, including constructing fine-grained dependency graph, handling communication overhead, and balancing workloads. HiSpTRSV addresses these challenges through dependency graph parsing, tile-based highly parallel algorithm, filtering mechanisms, and bidirectional matching with modular indexing. Experiments show that HiSpTRSV outperforms the state-of-the-art SpTRSV accelerator in terms of a 34.3% performance improvement. HiSpTRSV achieves a 3.58× speedup and 9.59× higher energy efficiency compared to GPUs.

**Index Terms**—Hardware accelerator, Computer systems organization, Reconfigurable computing.

## I. INTRODUCTION

SpTRSV is a fundamental computational operation for solving sparse triangular linear systems, expressed as  $L\vec{x} = \vec{y}$ , where  $L$  represents a sparse lower triangular matrix,  $\vec{x}$  is the dense vector to be solved, and  $\vec{y}$  is a known dense vector. SpTRSV is widely applied across various fields, including scientific computing [4], graph learning in machine learning [5], and dynamics analysis in physical simulations [6].

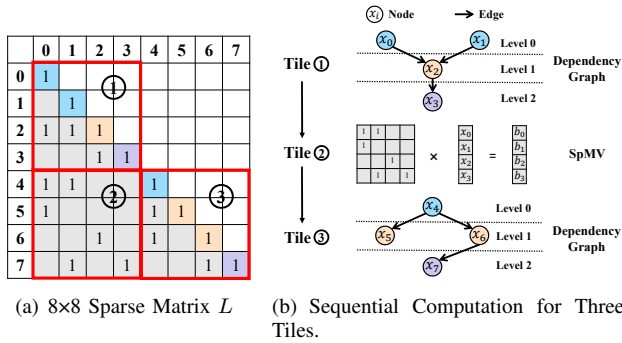


Fig. 1. Level-ST Algorithm Workflow.

In solving the SpTRSV problem, calculations have strict data dependencies: to compute an element  $x[i]$ , all the elements from  $x[0]$  to  $x[i-1]$  must first be computed. To address this challenge, researchers introduce the Level-Set algorithm to improve the parallelism of SpTRSV computations [15]. This algorithm represents matrix  $L$  as a dependency

graph with **nodes** and **edges**. It achieves faster computation by processing these components in parallel. Furthermore, when handling large-scale matrices, the matrix is typically divided into tiles, and the Level-Set [7] algorithm applies to each tile sequentially. Fig. 1 shows an example to explain this computational process. Fig. 1(a) shows an 8x8 sparse triangular matrix  $L$ , divided into 3 tiles (indicated by red boxes). Fig. 1(b) illustrates the sequential process for solving tiles 1, 2, and 3, where dependency graphs are generated for tiles 1 and 3 based on the Level-Set algorithm.

Taking tile 1 from Fig. 1(b) as an example, nodes  $x[0]$  and  $x[1]$  in Level 0 are first solved in parallel, and the associated edges (arrows, indicating node dependencies) are computed. Next, node  $x[2]$  in Level 1 is solved, along with its edges, followed by solving node  $x[3]$  in Level 2. When tile 1 is solved, the values of  $x[0]$  to  $x[3]$  are determined. Next, tile 2 perform Sparse Matrix-Vector Multiplication (SpMV) computation based on  $x[0]$  to  $x[3]$ . Finally, tile 3 is solved using results from tile 2 following similar steps.

Since the nodes and edges within tiles can be computed in parallel, researchers attempt to accelerate SpTRSV computation using FPGAs or GPUs [7]–[9]. However, these studies consider either inner-tile or inter-tile parallelism individually, failing to fully exploit SpTRSV’s parallel potential. Therefore, a natural question arises: can both inner-tile and inter-tile parallelism be achieved simultaneously? In fact, this is feasible. Fig. 2 illustrates the computation flow for this concept, we overlapped computations between tile 1 and tile 2 using a fine-grained pipelined design, realizing inter-tile parallelism. Additionally, nodes and edges within each tile are computed in parallel, realizing inner-tile parallelism. It should be noted that for larger matrices, more tiles can be computed in parallel. We will discuss this in detail in Section II.

However, implementing an accelerator with the aforementioned characteristics on FPGAs presents several challenges. **Dependency Parsing (C1):** Inter-tile parallelism requires more complex dependency graphs to describe the relationships, and randomness in matrix sparsity and size further increases the challenge in constructing the dependency graph. **Routing and Resources (C2):** Fine-grained computation requires more submodules, which not only increases on-chip resource consumption but also introduces additional communication overhead. Meanwhile, the interconnection between submodules further complicates the routing stage. **Load Balanc-**

\*The corresponding author is Fang Dong.

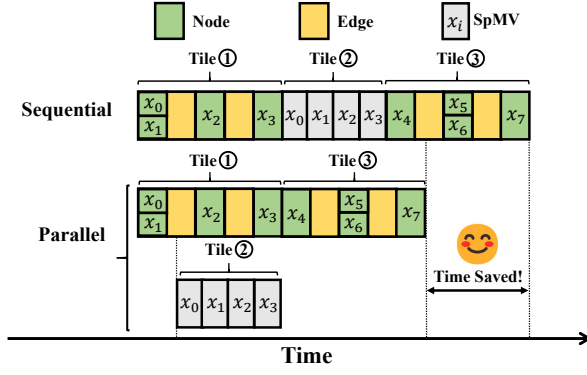


Fig. 2. Comparison of Computational Flows.

**ing (C3):** Randomness in sparse matrices makes the element count for each Processing Element (PE) uncertain, leading to imbalanced computational loads across PEs.

In this paper, we propose HiSpTRSV to address these challenges, with the following innovation:

- **Flexible Data Dependency Parsing:** We design a fine-grained data dependency graph to describe the dependencies in the SpTRSV solution process. This enables highly parallel computation. Additionally, HiSpTRSV enables automated data dependency graph construction for matrices of any size. This addresses challenge C1.
- **Fully Streamed High-Parallelism SpTRSV:** We develop a novel Level-Set algorithm that achieves high parallelism through fine-grained tile partitioning, unlocking SpTRSV's parallel potential. Within the module, resource overhead reduces through resource reuse. Between modules, routing complexity and communication overhead are reduced through stream-based communication and data filtering. This addresses challenge C2.
- **Mod-Based Bidirectional Index Match:** We design bidirectional indexing strategy based on modular arithmetic, which allocating elements to each PE, ensuring correct matching between dependent elements across modules, and achieving load balancing among PEs. This addresses challenge C3.

## II. MOTIVATION AND TILED HIGHLY PARALLELIZED LEVEL-SET ALGORITHM

In this section, we analyze the dependencies among different tiles in matrix  $L$  at a fine-grained level, demonstrating that these inter-tile dependencies are not strongly correlated. This observation creates opportunities for designing a highly parallel SpTRSV algorithm. For example, in Fig. 3(a), solving tile 2 depends on the SpMV result from tile 5, but it does not depend on tile 8. This means that the computations involving tile 2 and tile 8 can be carried out in parallel. Fig. 3 illustrates the analysis of inter-tile dependencies in the matrix, with the aim of implementing a highly parallel SpTRSV algorithm.

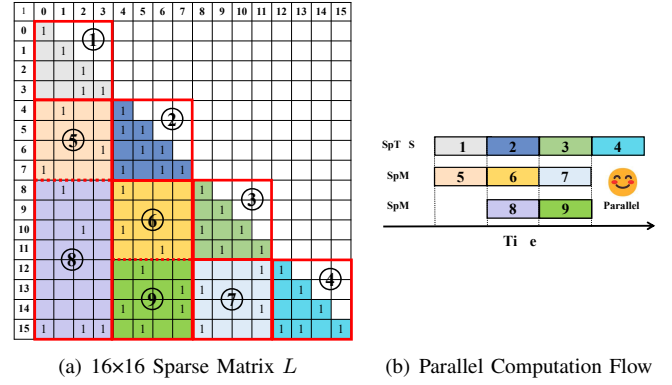


Fig. 3. High Parallelism via Dependency Decomposition

Fig. 3(a) presents a 16x16 lower triangular sparse matrix, which is divided into two types of tiles: tiles 1, 2, 3, and 4 represent the SpTRSV tiles, while tiles 5, 6, 7, 8, and 9 represent the SpMV tiles. When solving tile 1, tile 5 performs the SpMV computation in a pipelined parallel manner based on the results from tile 1, and stores the result on-chip. Once tile 1 is solved, tiles 2, 6, and 8 are computed in parallel. Similarly, tiles 3, 7, and 9 are computed in parallel, and once completed, the results of tiles 9 and 7 are accumulated. Finally, once tile 3 has completed its solution, tile 4 uses the stored result from tile 7 to proceed with its own computation. The computational flow is illustrated in Fig. 3(b), demonstrating that HiSpTRSV achieves higher parallelism.

Based on the above analysis, we design the Tiled High-Parallelism Level-Set Algorithm (THLS). This algorithm employs a tile-based strategy for computation, achieving high parallelism in SpTRSV while adhering to FPGA resource constraints. It enables full parallelism both inter-tile and inner-tile.

In the THLS algorithm, we organize the computation sequence for the SpTRSV and SpMV tiles as data dependency graphs  $G$  and  $M$ , with the computation step size at each stage managed by the *index*. The data dependency graph is allocated to  $P$  Processing Elements for parallel execution. Each PE executes SpTRSV computations (including node and edge computations) and SpMV computations in parallel, based on dependency graph indexing. When the tile completes, the PEs accumulate the results from the SpMV and SpTRSV tiles, storing the results on-chip in preparation for the next tile computation.

## III. HISPTRSV DESIGN AND IMPLEMENTATION

### A. HiSpTRSV Architecture Overview

As shown in Fig. 4, HiSpTRSV is a dataflow-based architecture that consists of multiple Processing Element Groups (PEGs), several data load modules, a data merge module, and a data write module. Each PEG connects to a High Bandwidth Memory (HBM) channel. The data load module is responsible for reading the data dependency graph and the dense vector  $\vec{y}$ , streaming this data to the PEGs. The data merge module consolidates the computation results from the various PEGs,

**Algorithm 1** Tiled High-Parallelism Level-Set Algorithm

**Require:** (1) SpTRSV Dependency Graph  $G$ , (2) SpMV Computation Graph  $M$ , (3) Dependency Graph Computation Index  $Index_G$ , (4) SpMV Region Computation Index  $Index_M$ , (5) Vertical Tiling Size  $W$ , (6) Matrix Dimension  $N$ , (7) Number of PEs  $P$ .

**Ensure:**  $L\vec{x} = \vec{y}$

```

1: for  $t = 0$  to  $N/W$  do
2:   Read  $y_t$  on chip
3:   for  $p = 0$  to  $P - 1$  parallel do
4:     Update  $layer$ 
5:     parallel do  $\triangleright$  SpTRSV and SpMV are parallel.
6:       for  $l = 0$  to  $layer - 1$  do  $\triangleright$  SpTRSV
7:          $(node, edge) \leftarrow Index_G[l]$ 
8:         for  $n = idx_g$  to  $idx_g + node - 1$  do
9:            $(row, col, val) \leftarrow G_{node}[n]$ 
10:           $x[row] \leftarrow (y_t[row] - y_{add}[row]) / val$ 
11:        end for
12:        Update  $idx_g$ 
13:        for  $e = idx_g$  to  $idx_g + edge - 1$  do
14:           $(row, col, val) \leftarrow G_{edge}[e]$ 
15:           $y_e[row] \leftarrow x[row] * val + y_e[row]$ 
16:        end for
17:      end for
18:       $m \leftarrow Index_M[l]$ 
19:      for  $s = idx_m$  to  $idx_m + m - 1$  do  $\triangleright$  SpMV
20:         $(row, col, val) \leftarrow M_{spmv}[s]$ 
21:         $y_s[row] \leftarrow x[row] * val + y_s[row]$ 
22:      end for
23:    end parallel
24:    Update  $idx_m$ 
25:     $y_{add} \leftarrow y_e + y_s$ 
26:  end for
27:  Write  $x_t$ 
28: end for

```

while the data write module handles writing the dense vector  $\vec{x}$  back to host memory.

Each PEG includes four core modules: computation module, filter module, cache module, and control module. The computation module includes the SpTRSV and SpMV modules, responsible for performing calculations in different tiles; the filter module filters out invalid elements during the synchronization process; the cache module manages the storage for intermediate computation results; and the control module serves as the core unit that oversees the operation of the PEG. We employ a fully streaming communication mechanism between the modules to optimize hardware resource utilization and simplify routing complexity.

In architecture-level communication, data synchronization between different PEGs is achieved through synchronous streams. We use a loopback data stream at the terminal PEG to return the synchronized data. This effectively implements a circular data synchronization stream among the PEGs, ensuring that each PEG receives the current solution  $\vec{x}$ , which

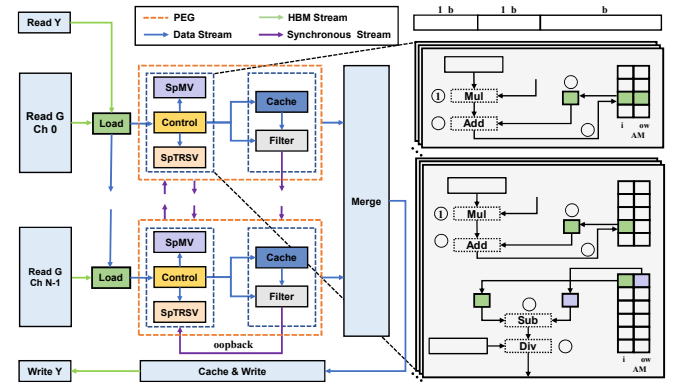


Fig. 4. HiSpTRSV Architecture and Computation Order for SpMV (Top) and SpTRSV (Bottom) Modules.

is required for the SpTRSV and SpMV computations in the next iteration. The dense vector  $\vec{y}$  is loaded through a single HBM channel and then distributed to each loading module via a chained broadcast mechanism.

### B. Processing Element Group (PEG) Design

To fully utilize HBM bandwidth, the PEG receives 512-bit wide data, containing 8 elements from the dependency graph. These 8 elements are distributed across 8 PEs, with each element using a 64-bit storage format that includes a 16-bit row index, a 16-bit column index, and a 32-bit value. The SpMV and SpTRSV serve as the core computational units in the PEG. The numerical labels in Fig. 4 indicate the execution sequence for the computation steps.

The SpMV module utilizes UltraRAM (URAM) units to store computation results, with the execution sequence comprising multiplication, read, addition, and write-back operations. Each URAM address is determined by the row index of the element. Once SpMV computations complete, data in URAM transfers in parallel to the SpTRSV module for accumulation. Given that the URAM unit is 72 bits wide, we divide the storage unit into two 36-bit regions (high and low) to store two accumulated values, optimizing the utilization of URAM resources. The design of Serpens and Sextans was referenced to accelerate the SpMV computation process [12], [18].

The SpTRSV module completes edge computations through steps similar to SpMV. The high region in URAM stores accumulation results from edge computations and SpMV computations, while the low region stores the  $\vec{y}$  values. When the computation dependent on  $x[i]$  is completed, the PEG first subtracts the  $Y_{add}$  value from the  $Y$  value. Then, it calculates  $x[i]$  by performing a division operation.

### C. Fine-Grained Data Dependency Graph

The dependency graph describes the SpTRSV computation process. We performed dependency analysis on matrix  $L$  at a fine-grained level and constructed a data dependency graph through tiling to support inter-tile parallel computation in SpTRSV. As shown in Fig. 5, we present an example data dependency graph for tiles 1, 5, and 8 from Fig. 3(a), with

row indices, column indices, and values labeled to correspond to elements in the sparse matrix.

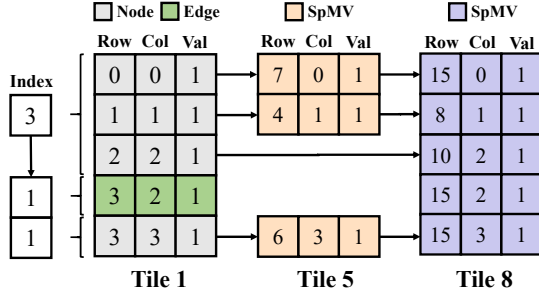


Fig. 5. Data Dependency Graph for Tiles 1, 5, and 8 in the 16x16 Matrix in Fig. 3(a).

The index list in Fig. 5 illustrates the solution process: First, three nodes are solved in parallel, followed by the computation of one edge, and finally, one node is solved. HiSpTRSV synchronously computes tile 1 and tile 5 based on dependency relationships. Once tile 1 is solved and the SpMV computation for tile 5 is complete, HiSpTRSV computes SpMV of tile 8 use dependency relationships and the values  $x[0]$  to  $x[3]$ . The computation results from tile 8 are stored on-chip to enable subsequent computations for other tiles. Notably, HiSpTRSV can generate dependency graphs for matrices of any size, ensuring flexibility in its application.

#### D. Data Filtering Unit

HiSpTRSV assigns elements to PEs through modulo operations on row indices. Due to the randomness of sparse matrices and element dependencies, some PEs may be temporarily idle. Taking the sparse matrix in Fig. 3(a) as an example, Fig. 6 illustrates the solution sequence for tiles 1 and 2 by two PEGs (assuming each PEG contains two PEs). The number is the row index for the element. Since node 3 depends on node 2, nodes 0, 1, and 2 are solved in parallel during the first solution cycle while PE3 remains idle. In the second cycle, only PE3 solves node 3 while other PEs are idle. To align PE computation lengths, "bubble elements" (invalid elements) are added to the dependency graph. It should be noted that HiSpTRSV consists of 16 PEGs, each containing 8 SpTRSV PEs and 4 SpMV PEs, resulting in a total of 192 PEs to compute in parallel. Although increasing the PE count expands the scale for parallel computation, it also raises the likelihood for bubble elements. Therefore, we design a data filtering unit to handle bubble elements.

During the node solving process, the solution results stored in URAM need to be synchronized. For example, when PE3 solves node 7, it requires the values of  $x[4]$  and  $x[6]$ , where  $x[4]$  is stored in the URAM of PEG0. Therefore, data synchronization is required to ensure each PEG can access all  $\vec{x}$  values within its own URAM. However, bubble elements introduce additional communication overhead to the synchronization process. We design a data filtering unit that identifies which elements should be written to URAM in parallel as PEGs perform write operations. When elements are

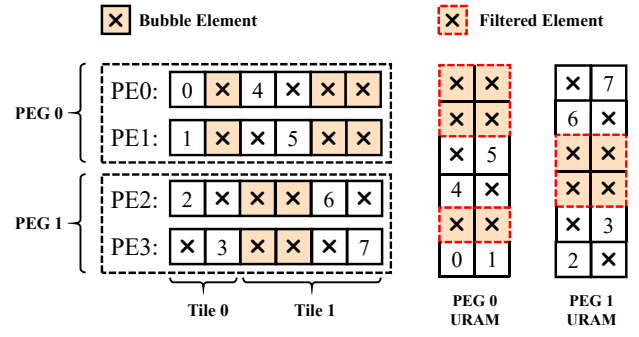


Fig. 6. Bubble Element Filtering Process.

bubble elements, URAM will reject storing them and record how many it successfully stores. This count represents the transmission length for data synchronization. Fig. 6 shows URAM-stored elements for PEG0 and PEG1, with red dashed lines indicating filtered elements that are neither stored in URAM nor participate in data synchronization.

#### E. Mod-Based Bidirectional Index Match

HiSpTRSV organizes the elements of SpMV and SpTRSV into a data dependency graph, assigns elements to PEs for computation, and ensures optimal load balancing. Fig. 7 shows the element allocation using modulo-based bidirectional matching.

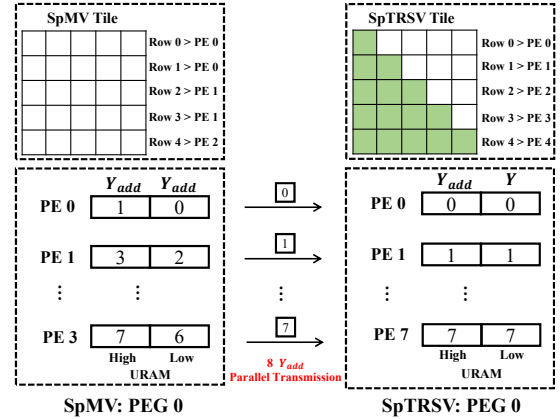


Fig. 7. Tiled SpMV and SpTRSV Element Mapping to PEs and URAM Layout.

For the elements in the SpMV tile, we assign two consecutive elements to the same PE because the URAM is divided into high and low regions to store accumulated values. The allocation method is:  $row/2\%N\_PEs$ , where  $N\_PEs$  represents the total number processing elements. For the elements in the SpTRSV region, only the high region of the URAM stores accumulated values. Meanwhile, one element is assigned to one PE at a time, and the allocation method is:  $row\%N\_PEs$ . It is worth noting that SpTRSV relies on the results generated by SpMV. Therefore, the SpMV units transfer the accumulated values  $Y_{add}$  in parallel to the SpTRSV units, using  $row$  as a common index to ensure the accumulated values match correctly between SpMV and SpTRSV.



TABLE I  
PERFORMANCE OF HiSPTRSV ON ALVEO U55C AND CUSPARSE ON NVIDIA V100 AND RTX3060

| Matrix Properties |                       |           |           |          | Performance (GFLOPS) |             |         |             | Improvement |
|-------------------|-----------------------|-----------|-----------|----------|----------------------|-------------|---------|-------------|-------------|
| ID                | Matrix Name           | Size      | NNZ       | Density  | RTX 3060             | NVIDIA V100 | LevelST | HiSpTRSV    | Percentage  |
| M1                | lp1                   | 534,388   | 1,088,904 | 3.81E-06 | 0.95                 | 0.68        | 1.71    | <b>2.48</b> | 45.0%       |
| M2                | parabolic_fem         | 525,825   | 2,100,225 | 7.60E-06 | 1.41                 | 1.21        | 4.02    | <b>4.82</b> | 19.9%       |
| M3                | com-Amazon_trig       | 334,863   | 718,060   | 1.63E-07 | 0.89                 | 0.51        | 2.07    | <b>2.78</b> | 34.3%       |
| M4                | gsm_106857_trig       | 589,446   | 2,932,426 | 8.44E-06 | 1.36                 | 1.37        | 3.62    | <b>4.81</b> | 32.9%       |
| M5                | hollywood-2009_trig   | 1,139,905 | 2,155,683 | 1.66E-06 | 0.88                 | 1.21        | 1.51    | <b>2.46</b> | 62.9%       |
| M6                | ins2_trig             | 309,412   | 1,392,214 | 1.45E-05 | 1.25                 | 0.61        | 2.81    | <b>3.86</b> | 37.4%       |
| M7                | c-big_trig            | 345,241   | 1,343,050 | 1.13E-05 | 1.71                 | 0.79        | 6.65    | <b>7.50</b> | 12.8%       |
| M8                | TSOPF_FS_b162_c4_trig | 40,798    | 791,443   | 4.39E-07 | 0.53                 | 0.39        | 2.63    | <b>3.52</b> | 33.8%       |
| M9                | ASIC_680k_trig        | 682,862   | 1,023,329 | 2.19E-06 | 0.082                | 0.077       | 0.61    | <b>0.88</b> | 44.3%       |
| M10               | hood                  | 220,542   | 5,494,489 | 1.13E-04 | 1.26                 | 1.63        | 5.11    | <b>6.10</b> | 19.4%       |
| M11               | roadNet-CA_trig       | 1,971,281 | 4,737,888 | 1.22E-06 | 0.35                 | 0.61        | 1.28    | <b>1.45</b> | 13.3%       |
| M12               | com-DBLP_trig         | 317,080   | 440,407   | 4.38E-06 | 0.81                 | 0.34        | 1.49    | <b>2.15</b> | 44.0%       |
| M13               | G3_circuit_trig       | 1,585,478 | 4,188,828 | 1.67E-06 | 0.075                | 0.21        | 0.92    | <b>1.26</b> | 37.0%       |
| M14               | tx2010_trig           | 914,231   | 5,579,489 | 6.68E-06 | 0.65                 | 0.63        | 1.57    | <b>1.89</b> | 20.4%       |
| M15               | net50                 | 16,320    | 480,760   | 1.81E-03 | 2.55                 | 0.41        | 4.01    | <b>6.68</b> | 66.6%       |
| M16               | c-73b                 | 169,422   | 724,348   | 2.52E-05 | 1.71                 | 0.57        | 4.28    | <b>5.89</b> | 37.6%       |

#### IV. EVALUATION

##### A. Experiment Setup

We design the circuit using AMD Vitis HLS C++ and implement the prototype with Vitis 2023.1. The experiments are mainly conducted on the Xilinx Alveo U55C platform [1], Nvidia RTX 3060 [2], and Tesla V100 [3]. The HLS code is developed based on the RapidStream TAPA [11] streaming framework, with layout optimization achieved through Autobridge [10]. The performance and power consumption are measured on actual hardware: GPU power is recorded using the NVML tool, while FPGA power is collected with the xbtutil tool during kernel execution. The measurement method for kernel execution time is kept consistent with LevelST [7] to ensure fair comparisons.

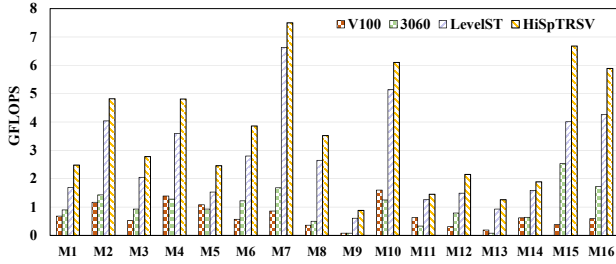


Fig. 8. FPGA and GPU Performance in GFLOPS.

We comprehensively compare our design against the state-of-the-art open-source SpTRSV accelerator LevelST [7] and the NVIDIA cuSparse library. The evaluation use 16 matrices from the SuiteSparse database [13], which are the same as those in the LevelST test set. Matrix sizes range from 16.3K to 1.97M rows, with non-zero elements between 440K and 5.58M. Tab. I lists detailed information for each matrix, including its size, the number of non-zero elements, and density. Matrix names with the suffix "trig" denote those derived through decomposition, while names without a suffix indicate directly extracted lower triangular matrices.

##### B. Overall Performance

The formula we use to compute GFLOPS is given by:

$$GFLOPS = \frac{2 \times NNZ}{10^9 \times \text{execution time}} \quad (1)$$

Here, NNZ represents the non-zero element count. We use GFLOPS as the primary performance metric, which directly reflects the computational efficiency and solving speed.

Tab. I provides a detailed list of the test results, with the last column showing the percentage improvement in performance compared to the LevelST. The 16 matrices exhibit throughput ranging from 0.88 to 7.50 GFLOPS. Compared to the state-of-the-art SpTRSV accelerator LevelST, HiSpTRSV achieves a 1.34x geometric mean speedup (up to 1.66x).

##### C. Communication

The random distribution in sparse matrix non-zero elements directly impacts the number of bubble elements within the dependency graph. The fewer the bubble elements, the lower the benefit of the filtering module. Fig. 9 shows the ratio of communication volume before and after using the filter module. Matrices M7, M15, and M16 display a relatively uniform pattern in non-zero element distribution, leading to balanced workloads across PEs and fewer bubble elements; consequently, communication volume does not decrease.

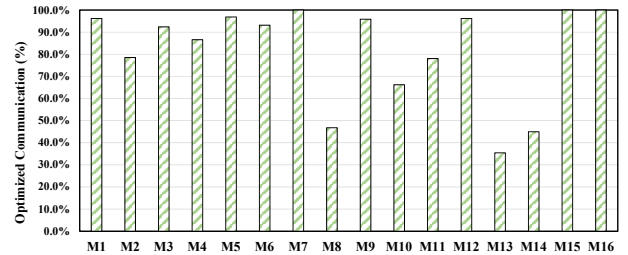


Fig. 9. Filter Module Optimization Ratio.

Conversely, matrices M8, M10, M11, M13, and M14 exhibit significant element clustering, with non-zero elements densely concentrated in specific rows. This distribution leads to severe load imbalance across PEs and generates numerous bubble elements, allowing the filter module to achieve significant optimization in these cases. The variation in optimization effectiveness not only confirms the design concept of the

filtering mechanism but also highlights the importance of matrix structure in parallel computation scheme design.

#### D. Resource Utilization and Design Frequency

Tab. II presents the resource utilization of HiSpTRSV on the U55C board. The SpTRSV module contains 8 PEs, while the SpMV module contains 4 PEs. We employ optimization directives to partition the Block RAM (BRAM) blocks within PEs to prevent BRAM conflicts between PEs, which implies that storing a single dense vector  $\vec{x}$  consumes 8 BRAM blocks. Since BRAM features dual ports and can serve two PEs in a single cycle, the SpTRSV and SpMV modules utilize 32 and 16 BRAM blocks, respectively.

TABLE II  
RESOURCE UTILIZATION OF HiSpTRSV PROTOTYPE ON XILINX U55C  
FPGA BOARD

| Resource | Available | Used    | Utilization (%) |
|----------|-----------|---------|-----------------|
| BRAM     | 2016      | 968     | 48.02           |
| DSP      | 9024      | 1220    | 13.52           |
| FF       | 2,607,360 | 821,979 | 31.53           |
| LUT      | 1,303,680 | 547,430 | 41.99           |
| URAM     | 960       | 640     | 66.67           |

URAMs are primarily utilized to cache intermediate results during computation in HiSpTRSV. The SpTRSV and SpMV modules each employ 8 URAM blocks, while the filter module, cache module, and write module collectively share 8 URAM blocks. Consequently, each PEG consumes a total of 40 URAM blocks. Given that the system incorporates 16 PEGs, the overall design occupies 640 URAM blocks. To fully utilize HBM's maximum bandwidth at 450 MHz with 512-bit ports, our kernels require only a frequency of 225 MHz [14]. HiSpTRSV operates at 246 MHz, successfully meeting this requirement.

#### E. Comparison to GPU

Fig. 8 compares the performance between Nvidia cuSparse running on 3060 and V100, and HiSpTRSV running on U55C. We select the better results between the 3060 and V100 for comparison with HiSpTRSV. For performance, HiSpTRSV achieve a geometric mean 3.58 $\times$  speedup. For energy efficiency, HiSpTRSV achieve a geometric mean 9.59 $\times$  improvement.

**3060 GPU vs. HiSpTRSV.** For performance, on average (geomean), HiSpTRSV is 4.10 $\times$  faster than the 3060 GPU (up to 16.80 $\times$ ). On the other hand, Fig. 10 compares their energy efficiency. HiSpTRSV achieves better energy efficiency than the 3060 across all matrices, showing a geomean improvement by 9.82 $\times$  (reaching up to 39.56 $\times$ ). When processing smaller sparse matrices, the RTX 3060 performs better due to its higher memory clock speed.

**V100 GPU vs. HiSpTRSV.** For performance, on average, HiSpTRSV is 5.44 $\times$  faster than the V100 GPU (up to 16.29 $\times$ ). The geomean energy efficiency improvement is 25.08 $\times$  (up to 59.94 $\times$ ). When processing larger sparse matrices, the V100 outperforms the RTX 3060 due to its larger on-chip memory and higher off-chip memory bandwidth.

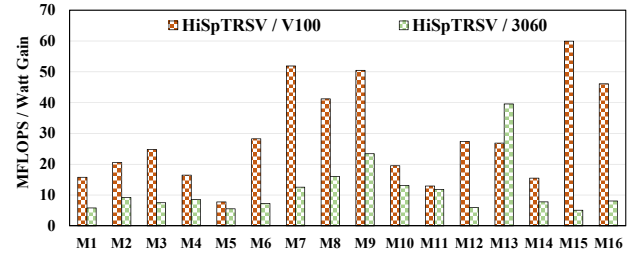


Fig. 10. Energy Efficiency Improvement of FPGA over GPU.

Due to the sparsity and dependencies in SpTRSV, some threads within the GPU remain idle during SpTRSV computations. However, these idle threads continue to consume power, leading to lower core utilization and higher energy consumption for the GPU. When handling sparse workloads, GPU peak utilization remains at only 0.6%, while average utilization reaches just 0.1% for workloads in [12], [16]. In addition, branch prediction and higher clock speeds significantly increase GPU kernel latency and power consumption [2], [17]. In contrast, we leverage FPGA reconfigurability to customize designs for specific applications, reducing additional latency and power consumption.

#### V. CONCLUSION

In this work, we develop the HiSpTRSV design to address the challenges faced in accelerating tile-based SpTRSV on HBM-based FPGAs. First, we implemented fine-grained dependency resolution at a granular level, enabling highly parallel computation in SpTRSV. Second, we develop a tile-based highly parallel SpTRSV algorithm and a stream-based filtering mechanism to achieve high parallel computation and low-overhead communication. Third, we design a modular bidirectional index matching mechanism to achieve balanced workload distribution. Moreover, HiSpTRSV can automatically construct and compute dependency graphs for matrices with any size. Experimental results demonstrate that our design significantly outperforms the state-of-the-art FPGA-based SpMV accelerator, LevelST, and Nvidia GPUs using cuSparse.

#### ACKNOWLEDGMENT

This work is supported by National Natural Science Foundation of China under Grants, No. 62232004, 62272101, Jiangsu Provincial Frontier Technology Research and Development Program under Grant BF2024070, Natural Science Foundation of Jiangsu Province under grant BK20230083, Shenzhen Key S&T Special Projects under Grant KJZD20240903100814018, Jiangsu Provincial Key Laboratory of Network and Information Security under Grants No.BM2003201, Key Laboratory of Computer Network and Information Integration of Ministry of Education of China under Grants No.93K-9, and partially supported by Collaborative Innovation Center of Novel Software Technology and Industrialization and Collaborative Innovation Center of Wireless Communications Technology. We also thank the Big Data Computing Center of Southeast University for providing the experiment environment and computing facility.

## REFERENCES

- [1] AMD, “Alveo U55C High Performance Compute Cards,” [Online]. Available: <https://www.amd.com/en/products/accelerators/alveo/u55c/a-u55c-p00g-pq-g.html>.
- [2] NVIDIA, “GeForce RTX 3060,” [Online]. Available: <https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/rtx-3060/>.
- [3] NVIDIA, “Tesla V100,” [Online]. Available: <https://www.nvidia.com/en-us/data-center/v100/>.
- [4] T. A. Davis and S. Rajamanickam, “An Efficient Multifrontal Solver for Sparse Matrices,” *SIAM Journal on Scientific Computing*, vol. 32, no. 2, pp. 521–544, 2010.
- [5] Y. Yu, J. Chen, T. Gao, and M. Yu, “DAG-GNN: DAG structure learning with graph neural networks,” in *Proceedings of the International Conference on Machine Learning*, 2019, pp. 7154–7163.
- [6] D. Negrut, R. Serban, and H. Mazhar, “Large-Scale Parallel Multi-Body Dynamics with Application to Vehicle Mobility Simulations,” *Journal of Computational and Nonlinear Dynamics*, vol. 9, no. 4, pp. 041010, 2014.
- [7] Z. He, L. Song, R. F. Lucas, and J. Cong, “LevelST: Stream-based Accelerator for Sparse Triangular Solver,” in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, Monterey, CA, USA, Mar. 2024, pp. 67–77.
- [8] Z. Lu and W. Liu, “TileSpTRSV: a tiled algorithm for parallel sparse triangular solve on GPUs,” *CCF Transactions on High Performance Computing*, vol. 5, pp. 129–143, 2023.
- [9] N. Ahmad, B. Yilmaz, and D. Unat, “A Split Execution Model for SpTRSV,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 11, pp. 2809–2822, Nov. 2021.
- [10] L. Guo, Y. Chi, J. Wang, J. Lau, W. Qiao, E. Ustun, Z. Zhang, and J. Cong, “AutoBridge: Coupling Coarse-Grained Floorplanning and Pipelining for High-Frequency HLS Design on Multi-Die FPGAs,” in *Proceedings of the 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, Monterey, CA, USA, Feb. 2021, pp. 229–240.
- [11] L. Guo, P. Maidee, Y. Zhou, C. Lavin, J. Wang, Y. Chi, W. Qiao, A. Kaviani, Z. Zhang, and J. Cong, “RapidStream: Parallel Physical Implementation of FPGA HLS Designs,” in *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, Monterey, CA, USA, Feb. 2022, pp. 106–117.
- [12] L. Song, Y. Chi, L. Guo, and J. Cong, “Serpens: A High Bandwidth Memory Based Accelerator for General-Purpose Sparse Matrix-Vector Multiplication,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, San Francisco, CA, USA, Jul. 2022, pp. 211–216.
- [13] T. A. Davis and Y. Hu, “The University of Florida Sparse Matrix Collection,” *ACM Transactions on Mathematical Software*, vol. 38, no. 1, pp. 1:1–1:25, 2011.
- [14] X. Chen, F. Cheng, H. Tan, Y. Chen, B. He, W.-F. Wong, and D. Chen, “Demystifying the Memory System of Modern Datacenter FPGAs for Software Programmers through Microbenchmarking,” in *Proceedings of the 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, Virtual Event, USA, Feb. 2021, pp. 24–35.
- [15] X. Wang, L. Liu, W. Xue, and L. Wu, “swSpTRSV: A fast sparse triangular solve with sparse level tile layout on Sunway architectures,” in *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, Vienna, Austria, Feb. 2018, pp. 338–350.
- [16] L. Song, L. Guo, S. Basalama, Y. Chi, R. F. Lucas, and J. Cong, “Callipepla: Stream centric instruction set and mixed precision for accelerating conjugate gradient solver,” in *Proceedings of the 2023 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, Monterey, CA, USA, Feb. 2023, pp. 247–258.
- [17] L. Guo, J. Lau, Z. Ruan, P. Wei, and J. Cong, “Hardware acceleration of long read pairwise overlapping in genome sequencing: A race between FPGA and GPU,” in *Proceedings of the 2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, San Diego, CA, USA, Apr. 2019, pp. 127–135.
- [18] L. Song, Y. Chi, A. Sohrabizadeh, Y.-k. Choi, J. Lau, and J. Cong, “Sextans: A streaming accelerator for general-purpose sparse-matrix dense-matrix multiplication,” in *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, Virtual Event, Feb. 2022, pp. 65–77.